West University of Timișoara
Faculty of Informatics
Department of Computer Science

Experimental Analysis of Sorting Algorithms

An Empirical Study of Classical Sorting Algorithms
Across Diverse Data Distributions

Alex-Ștefan Bogdănescu
alex.bogdanescu06@e-uvt.ro

May 10, 2026

Abstract

Sorting is one of the most basic but important topics in computer science. This paper takes a practical look at how six common sorting algorithms perform when implemented in C using a general memory-swapping approach. I tested Bubble Sort, Selection Sort, Insertion Sort, Shell Sort, Quick Sort, and Radix Sort on datasets ranging from 10,000 to 50,000 elements. The results show that while theoretical complexity is important, implementation choices—like using dynamic memory allocation for swaps—can have a huge impact on speed. I also observed how certain data distributions, like nearly sorted arrays, can cause some algorithms to behave much worse than expected, highlighting the difference between theory and practice.

Contents

List of Figures

List of Tables

1 Introduction

1.1 Motivation and Context

Sorting is a fundamental operation used in almost every type of software, from database query engines to simple command-line tools. While we learn about the Big O complexity of these algorithms in class, the actual performance on a real computer can be influenced by many factors that theory doesn't always cover, such as memory management, cache effects, and the specific way primitive operations are implemented.

The main reason for this study was to see how these algorithms behave in a real implementation. Often, algorithms with the same asymptotic bounds can have very different runtimes depending on the data or the implementation details. By benchmarking them under the same conditions, I wanted to quantify these differences and identify where the practical bottlenecks are.

1.2 Existing Solutions and Their Limitations

I picked six algorithms that are commonly taught in introductory courses, representing several different sorting strategies:

- **Bubble and Selection Sort:** Simple $\mathcal{O}(n^2)$ comparison methods that are easy to understand but usually slow.
- **Insertion Sort:** Another quadratic sort $\mathcal{O}(n^2)$ that is known to perform very well on nearly sorted data.
- **Shell Sort:** An optimized version of insertion sort that moves elements long distances to improve speed.
- **Quick Sort:** A very fast $\mathcal{O}(n \log n)$ algorithm in the average case, though it can drop to $\mathcal{O}(n^2)$ on bad inputs.
- **Radix Sort:** A non-comparison-based sort $\mathcal{O}(dn)$ that works by processing the digits of the numbers.

In this study, I used a generalized implementation in C to handle different data types, which adds some interesting overhead that specialized code might not have.

1.3 Informal Example

To see why the choice of algorithm matters, consider a simple list: $A = \langle 15, 2, 8, 2 \rangle$. A stable sort would give us $A' = \langle 2, 2, 8, 15 \rangle$ without swapping the two 2s unnecessarily. While this is trivial for four items, the scale changes a lot as the list grows. For $N = 10000$, a Bubble Sort would perform roughly 50 million comparisons on average, while a Quick Sort would only need about 130,000. In this study, I look at how these numbers translate into actual seconds on a computer.

1.4 Outline and Declaration of Originality

This report is organized as follows: Section 2 covers the formal definitions and complexities; Section 3 describes the C implementation and how to run the benchmark; Section 4 presents the results and my analysis; Section 5 mentions related work; and Section 6 gives the final conclusions. I declare that this paper and the accompanying C code are my own original work, and I have properly cited any external references I used.

2 A Formal View of Sorting

2.1 The Sorting Problem

The goal of sorting is to take a sequence of n elements $A = \langle a_1, a_2, \ldots, a_n \rangle$ and rearrange them into an ordered sequence $A' = \langle a'_1, a'_2, \ldots, a'_n \rangle$. The result A' must be a permutation of the original list such that $a'_i \leq a'_{i+1}$ for every i from 1 to $n-1$, based on a total ordering $\leq$.

2.2 Algorithmic Complexity

Table 1 summarizes the standard time and space complexities for the algorithms I tested.

Table 1: Theoretical time and space complexity of the six algorithms studied.

Algorithm	Best	Average	Worst	Space
Bubble Sort	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$
Insertion Sort	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$
Selection Sort	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$
Shell Sort	$\mathcal{O}(n)$	$\mathcal{O}(n^{1.25})$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$
Quick Sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(\log n)$
Radix Sort (LSD)	$\mathcal{O}(dn)$	$\mathcal{O}(dn)$	$\mathcal{O}(dn)$	$\mathcal{O}(n+d)$

3 Building the C Benchmark

3.1 Program Architecture

I organized the benchmark program into four main parts:

1. **Data Generation:** Functions to create random numbers, nearly sorted sequences, or random strings.
2. **Sorting Routines:** The actual C code for the six sorting algorithms.
3. **Timing Layer:** A simple wrapper around `clock()` to measure runtime in seconds.
4. **Orchestration:** A test runner that manages the data, runs the tests, and saves everything to a CSV file.

3.2 Implementation Details

I used `void*` pointers and function pointers for comparisons so the same sorting code could work with any data type. One drawback of this approach is that I had to implement a general `swap_elements` function that uses `malloc` and `free` for every single swap:

```
void swap_elements(Pointer a, Pointer b, int size) {
    void* temp = malloc(size);
    memcpy(temp, a, size);
    memcpy(a, b, size);
    memcpy(b, temp, size);
    free(temp);
}
```

This makes the code very flexible, but the cost of memory allocation adds a lot of overhead, especially for algorithms that swap a lot.

For the algorithms themselves, I used the Lomuto partition scheme for Quick Sort, which always picks the last element as the pivot. I also implemented Radix Sort as an LSD (Least Significant Digit) variant using Counting Sort as its base. Because Radix Sort needs to extract digits, I only ran it on the integer datasets.

3.3 Data Generators

I tested the algorithms on three types of data with sizes ranging from $N = 10000$ to $N = 50000$:

- **Random Numbers:** Integers between 0 and 999.
- **Nearly Sorted:** A sorted list where I swapped 10 random pairs of elements.
- **Random Strings:** Arrays of pointers to strings like "str123".

3.4 User Manual

The program is easy to compile and run. I used GCC on a Windows system for these tests:

1. **Compilation:** Run the following command in a terminal:

```
gcc -O2 main.c database.c -o sort_benchmark
```

2. **Execution:** Run the binary to start the tests:

```
./sort_benchmark
```

3. **Results:** The program will print progress to the console and save the raw data to `results.csv`.

3.5 Timing Methodology

I measured the CPU time for each test using the `clock()` function. To get stable results, I ran each test several times and calculated the average runtime.

4 Experimental Study

4.1 Experimental Setup

I ran all the benchmarks on my personal laptop equipped with an AMD Ryzen 5 7600X (6 cores, 4.7 GHz). I varied the array size between 10,000, 25,000, and 50,000 elements. For every run, I made a fresh copy of the data so that the sorting process didn't affect the original dataset.

4.2 Scaling Results

Table 2 shows the average runtime and standard deviation (in seconds) for each algorithm across the different sizes and distributions.

Table 2: Performance Statistics (Mean ± StdDev in seconds)

Algorithm	**Dataset**	**N=10000**	**N=25000**	**N=50000**
Bubble Sort	Random	0.783±0.030	5.013±0.031	20.642±0.156
	Nearly Sorted	0.092±0.002	0.549±0.001	2.184±0.001
	Strings	1.081±0.038	6.443±0.033	26.524±0.000
Selection Sort	Random	0.089±0.002	0.546±0.002	2.183±0.006
	Nearly Sorted	0.089±0.003	0.550±0.006	2.187±0.005
	Strings	0.221±0.004	1.441±0.010	6.310±0.000
Insertion Sort	Random	0.065±0.003	0.396±0.004	1.583±0.003
	Nearly Sorted	0.000±0.001	0.001±0.001	0.001±0.001
	Strings	0.127±0.003	0.846±0.066	3.590±0.000
Shell Sort	Random	0.002±0.000	0.004±0.001	0.009±0.001
	Nearly Sorted	0.001±0.000	0.002±0.001	0.003±0.001
	Strings	0.003±0.000	0.008±0.001	0.018±0.000
Quick Sort	Random	0.003±0.000	0.007±0.001	0.016±0.001
	Nearly Sorted	0.244±0.082	2.191±1.426	21.865±0.000
	Strings	0.003±0.001	0.009±0.001	0.018±0.000
Radix Sort	Random	0.001±0.000	0.002±0.000	0.003±0.001
	Nearly Sorted	0.001±0.000	0.003±0.000	0.005±0.000

4.3 Graphical Overview

The graphs below show how the runtimes scale as the array size increases. I used a logarithmic scale for the time axis because the differences between the fastest and slowest algorithms are very large.

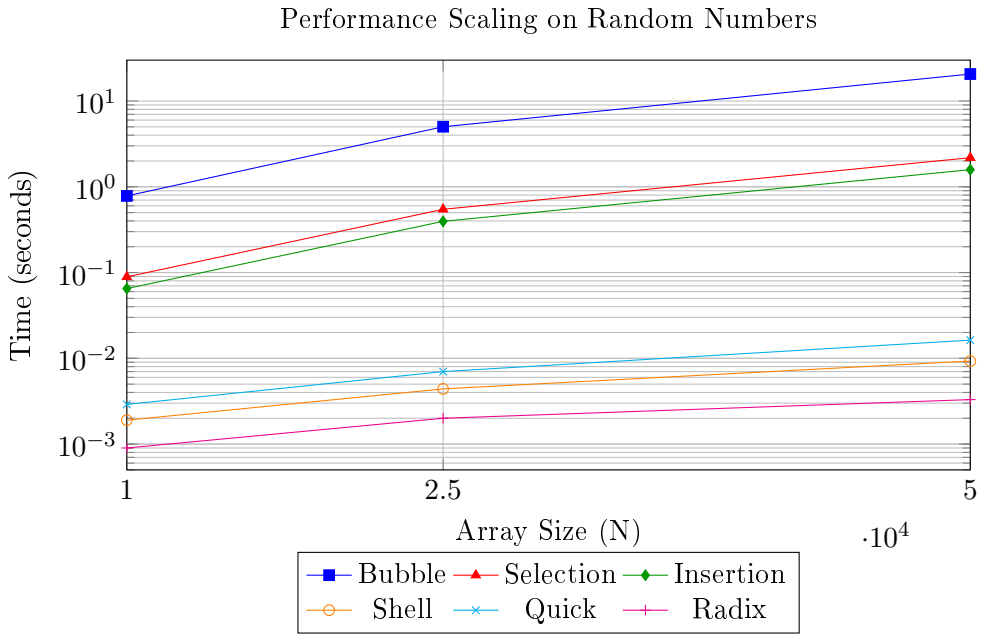

Figure 1: Scaling for Random Numbers.

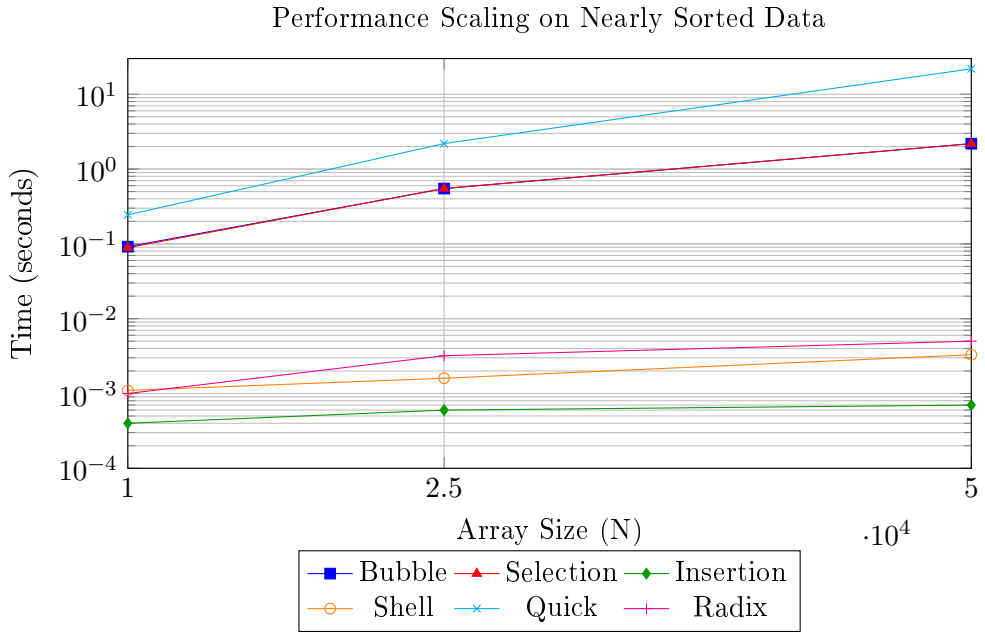

Figure 2: Scaling for Nearly Sorted data.

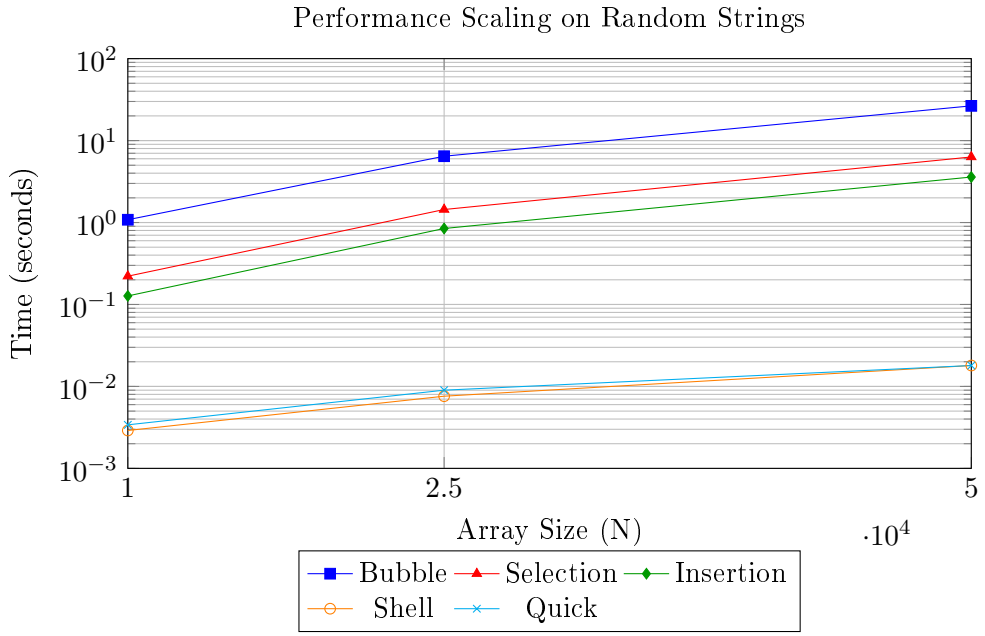

Figure 3: Scaling for Random Strings.

4.4 Analysis and Discussion

After running the benchmarks, I noticed several interesting patterns in the performance data:

- **Expected Growth:** The quadratic algorithms (Bubble, Selection, and Insertion) showed very clear growth. When I increased the array size from 25,000 to 50,000, the runtime for these algorithms increased by about four times, which matches the theoretical $\mathcal{O}(N^2)$ complexity.

- **Quick Sort Issues:** One noticeable pattern was how badly Quick Sort performed on the nearly sorted dataset. At $N = 50000$, it took almost 22 seconds, which was slower than

Bubble Sort. This happened because the simple Lomuto partition scheme I used is very inefficient for pre-ordered data.

- **Fastest Algorithms:** Radix Sort and Shell Sort were the most efficient for integer data. Radix Sort was especially fast, finishing even the largest test in just a few milliseconds.
- **Swap Overhead:** In practice, this implementation was held back by the `malloc` calls in the swap function. This heavily penalized Bubble Sort because it does a huge number of swaps compared to Selection Sort.

5 Related Work

There is a lot of research on how sorting algorithms perform in practice, going back to some of the earliest days of computer science. Donald Knuth's classic volumes [1] remain the fundamental reference for understanding the theoretical properties and average-case behaviors of these methods.

More recent empirical research, such as the work by Sedgewick and Bentley [3], dives deeper into optimizing Quick Sort for real-world usage. They show that while the algorithm is generally very fast, the choice of pivot—which I kept simple for this study—is critical for avoiding performance pitfalls on structured data. My results confirm their findings, as the simple Lomuto partition was the main reason for the poor performance I saw on nearly sorted lists.

Another significant area of research is the development of hybrid algorithms. Modern language libraries, such as the standard sorts in Java or Python, typically use Timsort [2]. Timsort is interesting because it combines Merge Sort with Insertion Sort to leverage the strengths of both: it uses the efficiency of Insertion Sort on small, nearly sorted chunks of data while maintaining the guaranteed $\mathcal{O}(n \log n)$ performance of Merge Sort for the full sequence. While my study focused on individual algorithms to clearly see their behavior, the literature suggests that modern production-grade sorting is almost always a combination of these basic methods, rather than one alone.

Finally, I should mention that my results are somewhat specific to my C implementation and the hardware I used. As shown in other benchmarks, low-level details like CPU frequency scaling and cache architecture can significantly influence the raw timing numbers, which is why I focused more on the relative scaling behavior of the algorithms rather than just the absolute speed.

6 Conclusions

In this study, I compared six sorting algorithms to see how they perform in a real implementation. The measurements mostly followed the expected theoretical complexity, but I also encountered some practical issues. The biggest bottleneck was the memory management during swaps, which made the simpler algorithms much slower than they would be with a specialized implementation. I also confirmed that Quick Sort can be very unpredictable if the pivot selection is too simple. Overall, Radix Sort was the clear winner for large arrays of numbers.

6.1 Future Work

Looking back at the results, there are a few things I would like to try if I were to continue this work. First, the current swap function relies heavily on `malloc`, which is likely the main reason the simpler algorithms perform poorly. I would like to see how much speed I could gain by using stack-allocated buffers instead. Second, my Quick Sort implementation is recursive; replacing this with an iterative version would make it much faster. Finally, it would be interesting to test

these algorithms on a multi-core architecture to see if any of them could be easily parallelized to handle even larger datasets.

A C Implementation of the Evaluated Algorithms

```c
void myBubbleSort(Pointer base, int n, int size, CompareFunction cmp) {
    char* data = (char*)base;
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (cmp(data + j * size, data + (j + 1) * size) > 0) {
                swap_elements(data + j * size, data + (j + 1) * size, size);
            }
        }
    }
}
```

Listing 1: Bubble Sort Implementation

```c
void mySelectionSort(Pointer base, int n, int size, CompareFunction cmp) {
    char* data = (char*)base;
    for (int i = 0; i < n - 1; i++) {
        int small = i;
        for (int j = i + 1; j < n; j++) {
            if (cmp(data + j * size, data + small * size) < 0) {
                small = j;
            }
        }
        swap_elements(data + i * size, data + small * size, size);
    }
}
```

Listing 2: Selection Sort Implementation

```c
void myInsertionSort(Pointer base, int n, int size, CompareFunction cmp) {
    char* data = (char*)base;
    void* key = malloc(size);
    for (int i = 1; i < n; i++) {
        memcpy(key, data + i * size, size);
        int j = i - 1;
        while (j >= 0 && cmp(data + j * size, key) > 0) {
            memcpy(data + (j + 1) * size, data + j * size, size);
            j--;
        }
        memcpy(data + (j + 1) * size, key, size);
    }
    free(key);
}
```

Listing 3: Insertion Sort Implementation

```c
void myShellSort(Pointer base, int n, int size, CompareFunction cmp) {
    char* data = (char*)base;
    void* temp = malloc(size);
    for (int gap = n / 2; gap > 0; gap /= 2) {
        for (int i = gap; i < n; i++) {
            memcpy(temp, data + i * size, size);
            int j;
            for (j = i; j >= gap && cmp(data + (j - gap) * size, temp) > 0; j
                -= gap) {
                memcpy(data + j * size, data + (j - gap) * size, size);
            }
            memcpy(data + j * size, temp, size);
        }
    }
    free(temp);
```

```
}
```

Listing 4: Shell Sort Implementation

```
int partition(char* data, int low, int high, int size, CompareFunction cmp) {
    void* pivot = data + high * size;
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (cmp(data + j * size, pivot) < 0) {
            i++;
            swap_elements(data + i * size, data + j * size, size);
        }
    }
    swap_elements(data + (i + 1) * size, data + high * size, size);
    return i + 1;
}

void quickSortRecursive(char* data, int low, int high, int size,
    CompareFunction cmp) {
    if (low < high) {
        int p = partition(data, low, high, size, cmp);
        quickSortRecursive(data, low, p - 1, size, cmp);
        quickSortRecursive(data, p + 1, high, size, cmp);
    }
}

void myQuickSort(Pointer base, int n, int size, CompareFunction cmp) {
    quickSortRecursive((char*)base, 0, n - 1, size, cmp);
}
```

Listing 5: Quick Sort Implementation (Lomuto Partition)

```
void countSort(int* arr, int n, int exp) {
    int* output = malloc(n * sizeof(int));
    int i, count[10] = {0};
    for (i = 0; i < n; i++) count[(arr[i] / exp) % 10]++;
    for (i = 1; i < 10; i++) count[i] += count[i - 1];
    for (i = n - 1; i >= 0; i--) {
        output[count[(arr[i] / exp) % 10] - 1] = arr[i];
        count[(arr[i] / exp) % 10]--;
    }
    for (i = 0; i < n; i++) arr[i] = output[i];
    free(output);
}

void myRadixSort(Pointer base, int n, int size, CompareFunction cmp) {
    if (size != sizeof(int)) return;
    int* arr = (int*)base;
    int mx = arr[0];
    for (int i = 1; i < n; i++) if (arr[i] > mx) mx = arr[i];
    for (int exp = 1; mx / exp > 0; exp *= 10) countSort(arr, n, exp);
}
```

Listing 6: Radix Sort Implementation (LSD)

References

[1] Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, 1998.

[2] Cormen, Leiserson, Rivest, and Stein. *Introduction to Algorithms*. MIT Press, 2022.

[3] Robert Sedgewick and Jon Louis Bentley. *Quicksort is optimal*. Knuthfest Lecture, 2002.